

INSO & MAIS



Procesamiento de datos

Miguel Ángel Fernández Díaz
2024-2025



Tema 2:

Spark básico

1. Introducción a Spark

La persistencia

- Uso intensivo de disco
- Memoria infrautilizada



Implementación de MapReduce

- Fase de Shuffle
 - Redistribución irregular de datos
 - Sin localidad del dato

Cálculos iterativos

- Cada iteración = Nuevo trabajo → Overhead
- Disco para datos entre etapas
- Redundancia de disco para fases de Map (escritura) y Reduce (lectura)

- Es un motor de procesamiento para entornos Big Data como Data Lakes que aprovecha el procesamiento en memoria distribuido para realizar análisis de datos de forma eficiente y con un alto rendimiento para grandes volúmenes de datos
- Es capaz de acceder a prácticamente cualquier fuente de datos
- Es uno de los proyectos más populares dentro del Apache Foundation
- Incluido en las plataformas más importantes de Big Data como Cloudera y Stratio, así como en servicios cloud como Amazon, Microsoft y Google
- Proyecto de código abierto, sostenido por Databricks principalmente, y que se ejecuta en la JVM



¿Por qué Spark?

	Hadoop World Record	Spark 100 TB *	Spark 1 PB
Data Size	102.5 TB	100 TB	1000 TB
Elapsed Time	72 mins	23 mins	234 mins
# Nodes	2100	206	190
# Cores	50400	6592	6080
# Reducers	10,000	29,000	250,000
Rate	1.42 TB/min	4.27 TB/min	4.27 TB/min
Rate/node	0.67 GB/min	20.7 GB/min	22.5 GB/min
Sort Benchmark Daytona Rules	Yes	Yes	No
Environment	dedicated data center	EC2 (i2.8xlarge)	EC2 (i2.8xlarge)

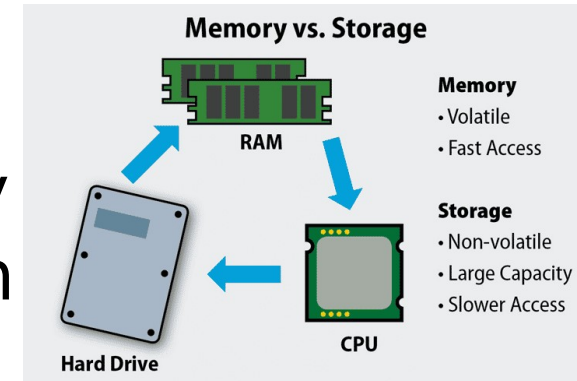
* not an official sort benchmark record

Competición Daytona Grey Sort (2014)

- Originalmente desarrollado por la UC Berkeley en 2009 como proyecto de investigación de Matei Zahara (AMPLab)
- Surgió como “ejemplo” de framework sobre Apache Mesos
- Su primera aplicación fue para monitorizar y predecir el tráfico en la Bahía de San Francisco
- En 2013, su incipiente comunidad le lleva a formar parte del programa Apache incubator
- En 2014, su estabilidad y ritmo de desarrollo le lleva a convertirse en un proyecto Apache Top Level

¿Por qué surge?

- Nace como alternativa a la lentitud en algunos procesos Map-Reduce de Hadoop y como herramienta con una librerías más amigable y más transparente para los desarrolladores
- Los procesos iterativos e interactivos resultaban muy lentos dado que los cálculos intermedios se persistían en disco
- Se puede apoyar en disco para persistencia temporal pero nunca para procesamiento
- La desaceleración en la velocidad de los procesos hizo que muchos esfuerzos se centrasen en distribuir el procesamiento paralelamente entre varios nodos de computación
- Nace como necesidad de un framework con tolerancia a fallos automática



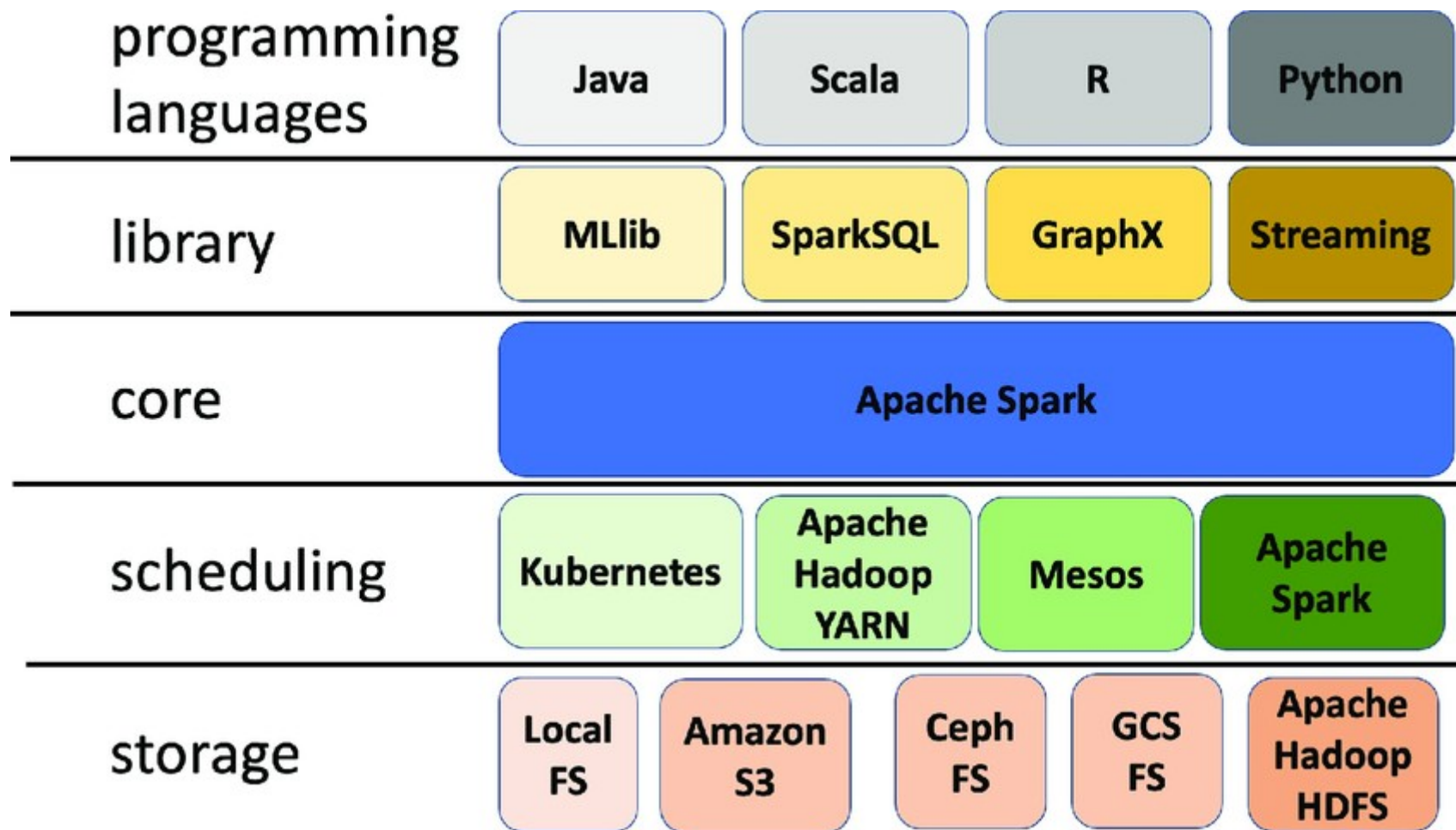
```
1 public class WordCount {
2     public static class TokenizerMapper
3         extends Mapper<Object, Text, Text, IntWritable>{
4
5         private final static IntWritable one = new IntWritable(1);
6         private Text word = new Text();
7
8         public void map(Object key, Text value, Context context
9             ) throws IOException, InterruptedException {
10             StringTokenizer itr = new StringTokenizer(value.toString());
11             while (itr.hasMoreTokens()) {
12                 word.set(itr.nextToken());
13                 context.write(word, one);
14             }
15         }
16     }
17
18     public static class IntSumReducer
19         extends Reducer<Text, IntWritable, Text, IntWritable> {
20         private IntWritable result = new IntWritable();
21
22         public void reduce(Text key, Iterable<IntWritable> values,
23             Context context
24             ) throws IOException, InterruptedException {
25             int sum = 0;
26             for (IntWritable val : values) {
27                 sum += val.get();
28             }
29             result.set(sum);
30             context.write(key, result);
31         }
32     }
33
34     public static void main(String[] args) throws Exception {
35         Configuration conf = new Configuration();
36         String[] otherArgs = new GenericOptionsParser(conf, args).getRemainingArgs();
37         if (otherArgs.length < 2) {
38             System.err.println("Usage: wordcount <in> [<in>...] <out>");
39             System.exit(2);
40         }
41         Job job = new Job(conf, "word count");
42         job.setJarByClass(WordCount.class);
43         job.setMapperClass(TokenizerMapper.class);
44         job.setCombinerClass(IntSumReducer.class);
45         job.setReducerClass(IntSumReducer.class);
46         job.setOutputKeyClass(Text.class);
47         job.setOutputValueClass(IntWritable.class);
48         for (int i = 0; i < otherArgs.length - 1; ++i) {
49             FileInputFormat.addInputPath(job, new Path(otherArgs[i]));
50         }
51         FileOutputFormat.setOutputPath(job,
52             new Path(otherArgs[otherArgs.length - 1]));
53         System.exit(job.waitForCompletion(true) ? 0 : 1);
54     }
55 }
```

```
1 val f = sc.textFile(inputPath)
2 val w = f.flatMap(l => l.split(" ")).map(word => (word, 1)).cache()
3 w.reduceByKey(_ + _).saveAsText(outputPath)
```

WordCount in 3 lines of Spark

WordCount in 50+ lines of Java MR

Plataforma de computación distribuida de propósito general, que permite combinar, por ejemplo, sentencias SQL con algoritmos de Machine learning, gracias a su motor unificado de análisis



Spark SQL

- Datos estructurados

Spark Streaming

- Aplicaciones tolerante a fallos en streaming

MLlib

- Librería machine learning escalable

ML Pipelines

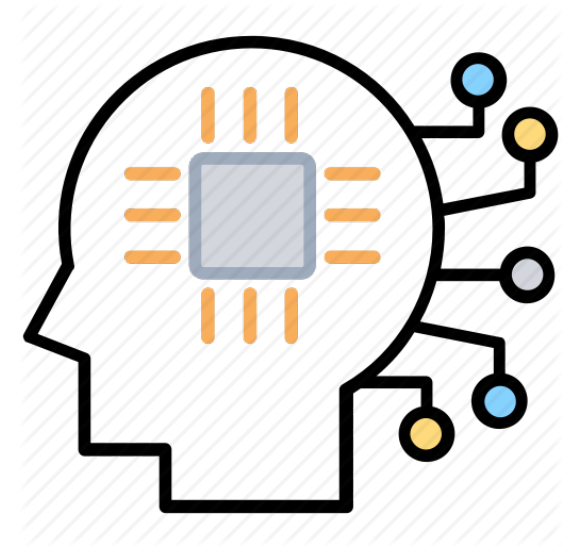
- API de alto nivel para pipelines Machine learning sobre DataFrames

GraphX

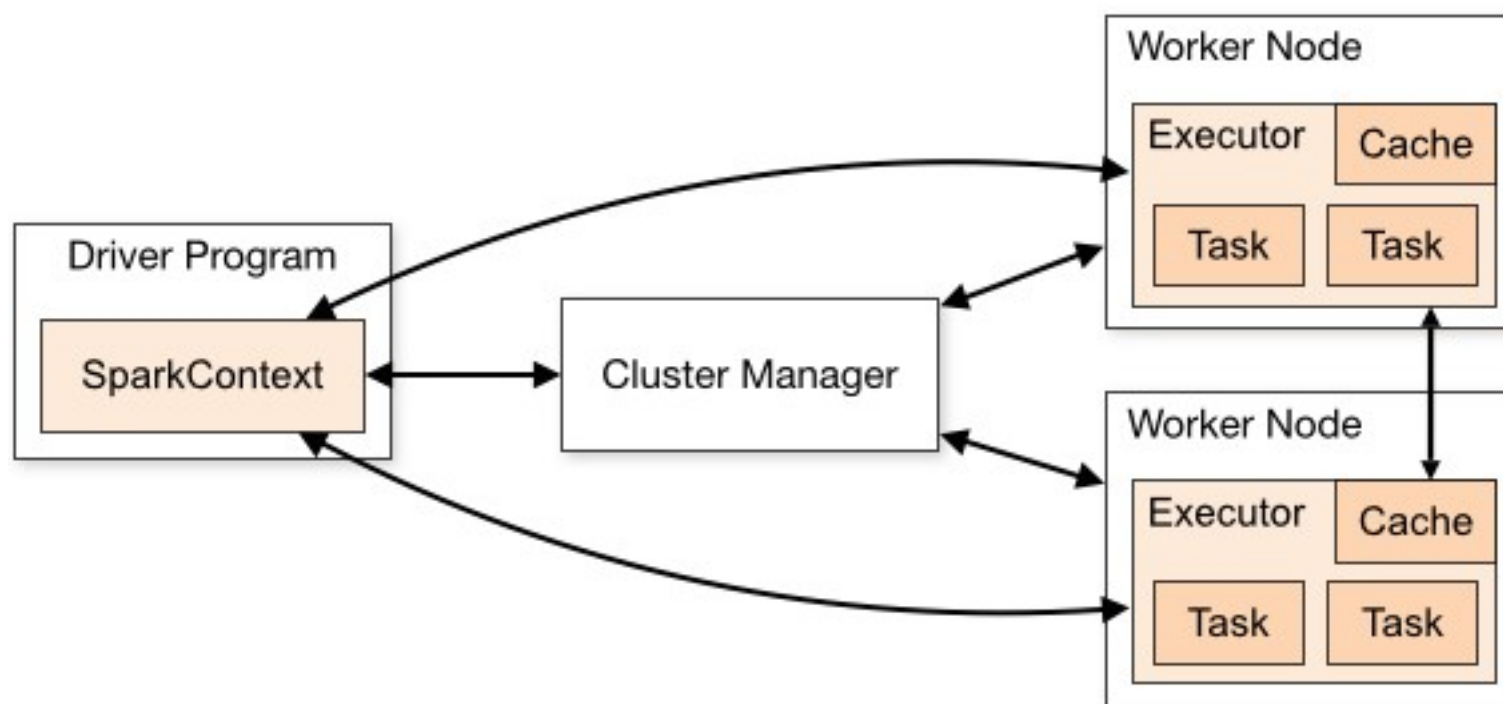
- API para computación de grafos en paralelo

SparkR

- Paquete de R package con una interfaz ligera



- Una aplicación de Spark se ejecuta como un conjunto independiente de procesos que corren en un cluster y que están coordinados gracias al SparkContext
- El Cluster Manager se encarga de asignar recursos a las diferentes aplicaciones
- Aplicaciones aisladas: cada aplicación tiene su propio conjunto de ejecutores



El parámetro master indica donde se encuentra el master del cluster de Spark. Además puede ayudarnos a crear cluster locales.

<i>master</i>	<i>description</i>
local	run Spark locally with one worker thread (no parallelism)
local[K]	run Spark locally with K worker threads (ideally set to # cores)
spark://HOST:PORT	connect to a Spark standalone cluster; PORT depends on config (7077 by default)
mesos://HOST:PORT	connect to a Mesos cluster; PORT depends on config (5050 by default)

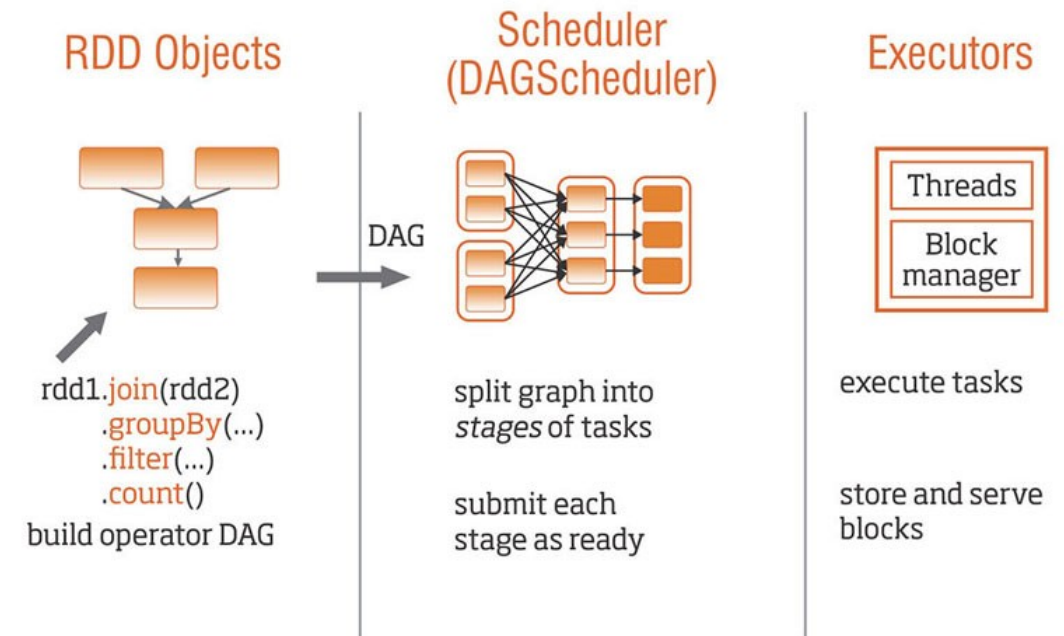
- Master:

- El Máster gestiona los recursos de las diferentes aplicaciones
- Adquiere executors en los nodos del cluster los cuales procesan la información y gestionan las caches

- Driver:

- Se conecta al cluster manager para levantar el cluster
- Distribuye el código entre los executors
- Envía las tareas de ejecución a los executors

Job Scheduling Process



Resilient Distributed Dataset

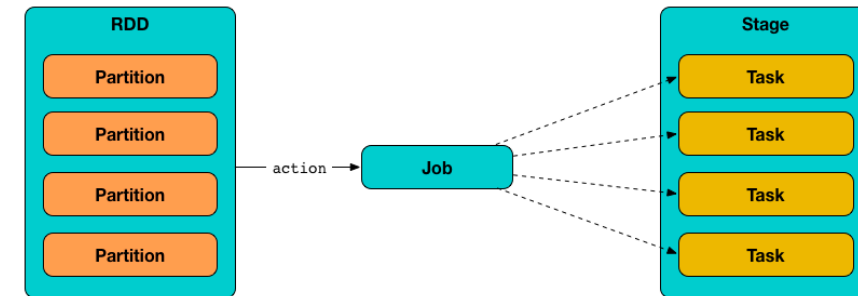
- Abstracción de datos principal de Spark Core
- Reparte datos inmutables entre las particiones y los nodos del cluster para poder operar en paralelo
- Tolerante a fallos

► Se recupera automáticamente de fallos en los nodos gracias a un grado de linaje realizando una reconstrucción

- Intenta aprovechar la localidad del dato
- Óptimo para algoritmos iterativos
- Concepto nacido a raíz del paper:

[Resilient Distributed Datasets: A Fault-Tolerant Abstraction for In-Memory Cluster Computing](#)

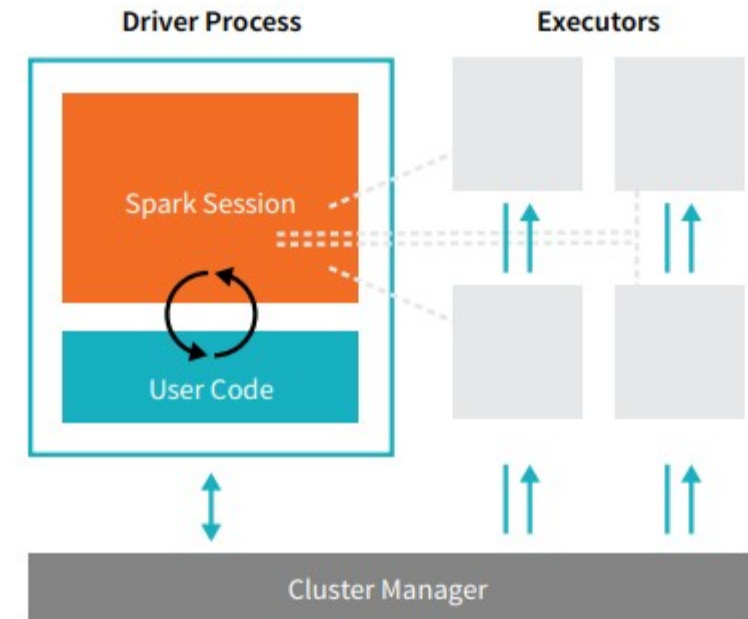
- Abstrae, parcialmente, al usuario de la complejidad de la distribución y procesamiento en paralelo



A pesar de que Batch Processing es un modelo restringido a una serie de casos de uso por defecto, gracias a los RDDs se puede utilizar en multitud de aplicaciones

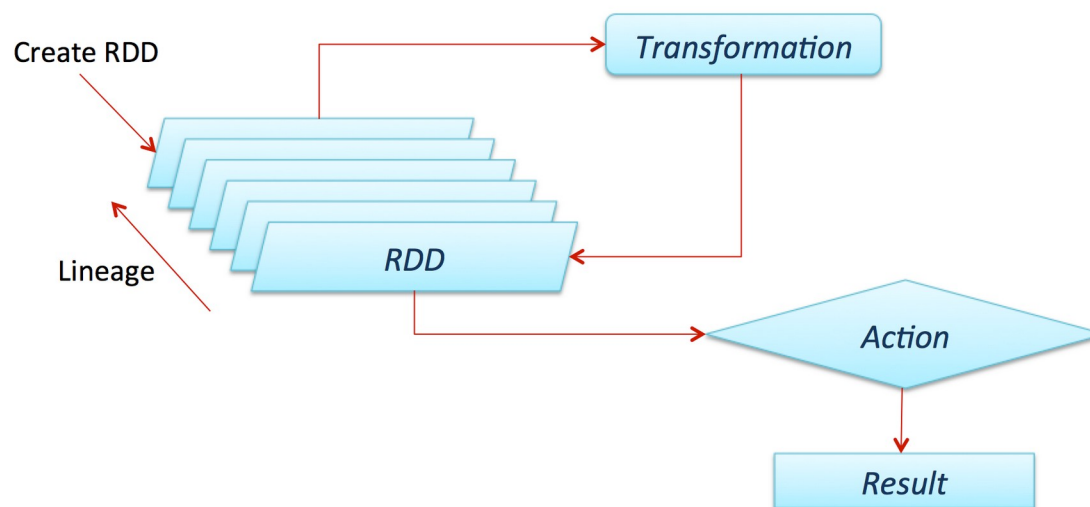
Se genera a partir de 2 escenarios:

- Colección que existe en el driver y que se distribuye entre los nodos del cluster
- Datos que residen en una fuente de datos externa



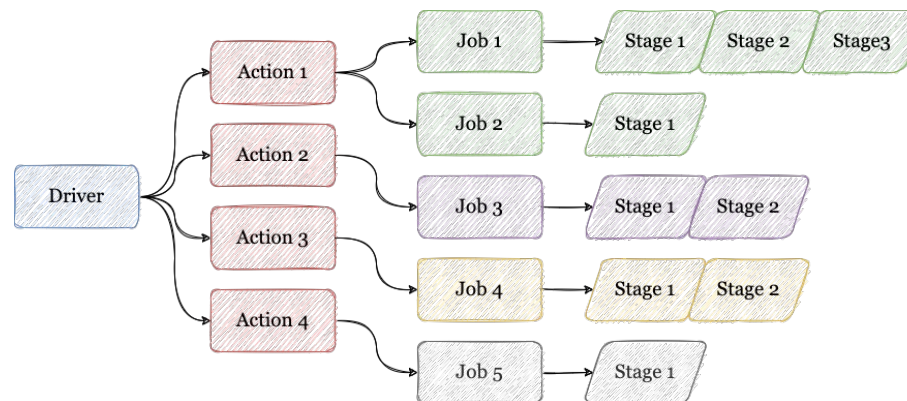
Las **transformaciones** son operaciones “lazy” (perezosas) y, por tanto, afectan solo al Driver y nunca a los Executors

- No calculan los resultados de inmediato
- Sólo recuerda las transformaciones aplicadas a algún conjunto de datos de base (por ejemplo, un archivo)
- <https://spark.apache.org/docs/3.3.4/rdd-programming-guide.html#transformations>
- Ejemplos: Map, Filter, Sample, Union, GroupByKey, ReduceByKey, Join, Cache



Las **acciones** desencadenan la ejecución de tareas en los Executors

- Las transformaciones se calculan sólo cuando una acción requiere un resultado para ser devuelto al driver
- Este diseño permite a Spark funcionar de manera más eficiente
 - Realizar lógica incremental sobre el conjunto de datos hasta que obtengamos una serie de transformaciones que devolverán un conjunto de datos pequeño
- <https://spark.apache.org/docs/latest/rdd-programming-guide.html#actions>
- Ejemplos: Reduce, Collect, Count, Save, LookupKey



```
// base RDD
val lines = sc.textFile("dummy.log")
// transformed RDDs
val errors = lines.filter(_.startsWith("ERROR"))
val messages = errors.map(_.split("\t")).map(r => r(1))
messages.cache()
// action 1
messages.filter(_.contains("mysql")).count()
// action 2
messages.filter(_.contains("php")).count()
```

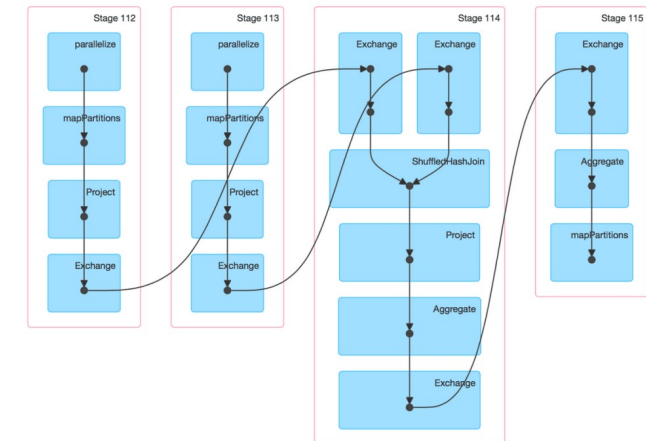
Details for Job 8

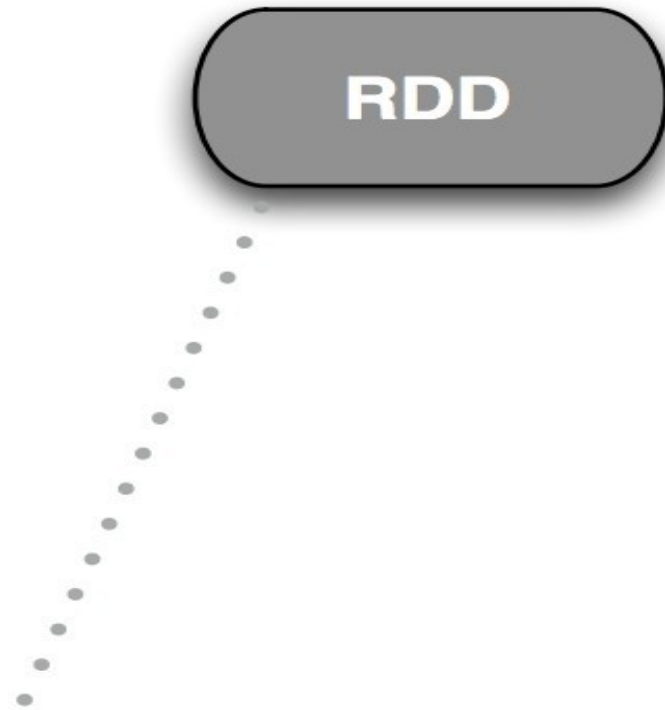
Status: SUCCEEDED

Completed Stages: 4

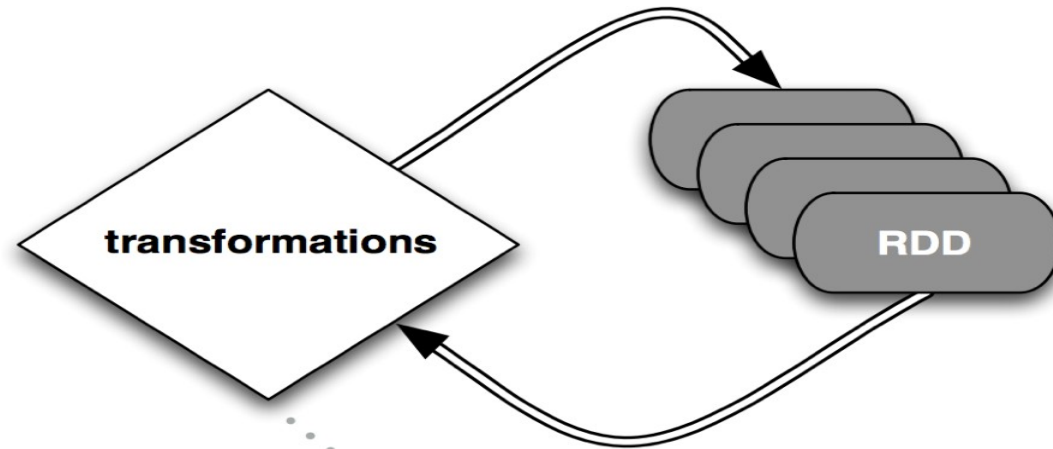
▶ Event Timeline

▼ DAG Visualization

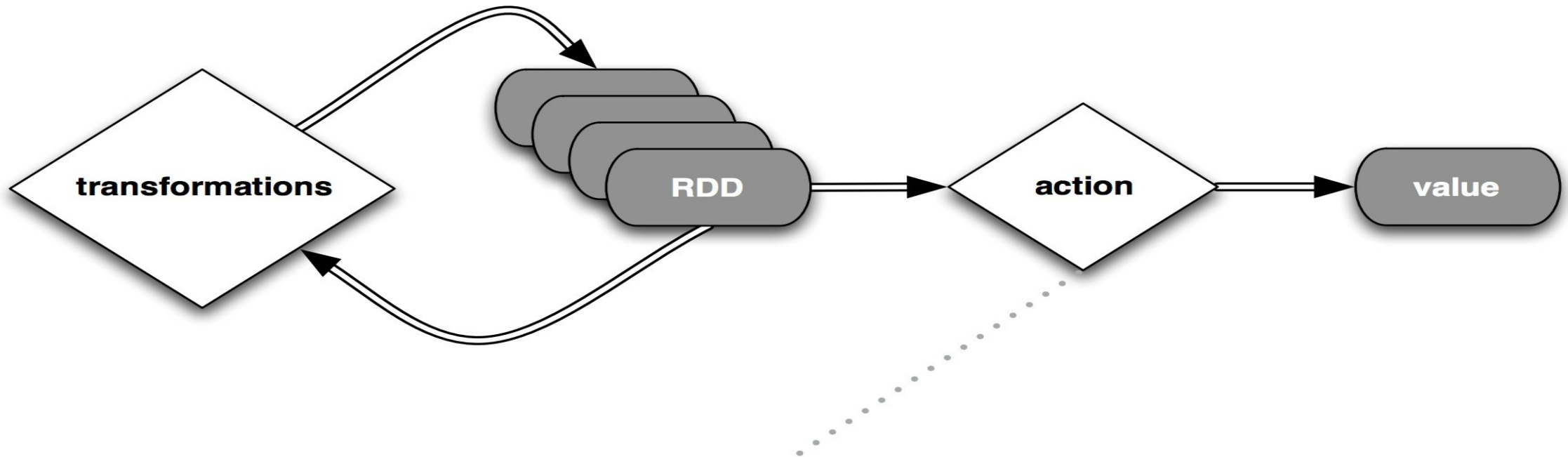




```
// base RDD  
val lines = sc.textFile("hdfs://...")
```



```
// transformed RDDs  
val errors = lines.filter(_.startsWith("ERROR"))  
val messages = errors.map(_.split("\t")).map(r => r(1))  
messages.cache()
```



```
// action 1  
messages.filter(_.contains("mysql")).count()
```

`rdd.cache()`, `rdd.persist()` → `rdd.unpersist()`

Storage Level	Meaning
MEMORY_ONLY	Store RDD as deserialized Java objects in the JVM. If the RDD does not fit in memory, some partitions will not be cached and will be recomputed on the fly each time they're needed. This is the default level.
MEMORY_AND_DISK	Store RDD as deserialized Java objects in the JVM. If the RDD does not fit in memory, store the partitions that don't fit on disk, and read them from there when they're needed.
MEMORY_ONLY_SER (Java and Scala)	Store RDD as <i>serialized</i> Java objects (one byte array per partition). This is generally more space-efficient than deserialized objects, especially when using a fast serializer , but more CPU-intensive to read.
MEMORY_AND_DISK_SER (Java and Scala)	Similar to MEMORY_ONLY_SER, but spill partitions that don't fit in memory to disk instead of recomputing them on the fly each time they're needed.
DISK_ONLY	Store the RDD partitions only on disk.
MEMORY_ONLY_2, MEMORY_AND_DISK_2, etc.	Same as the levels above, but replicate each partition on two cluster nodes.
OFF_HEAP (experimental)	Similar to MEMORY_ONLY_SER, but store the data in off-heap memory . This requires off-heap memory to be enabled.

- Broadcast
 - `scala> val broadcastVar = sc.broadcast(Array(1,2,3))`
 - `broadcastVar:`
`org.apache.spark.broadcast.Broadcast[Array[Int]]`
`= Broadcast(2)`
 - `scala> broadcastVar.value`
 - `res6: Array[Int] = Array(1,2,3)`
- Accumulators
 - `scala> val accum = sc.accumulator(0)`
`accum: org.apache.spark.Accumulator[Int] = 0`
 - `scala> sc.parallelize(Array(1,2,3,4)).foreach(x => accum +=x)`
 - `scala> accum.value`
 - `res8: Int =10`

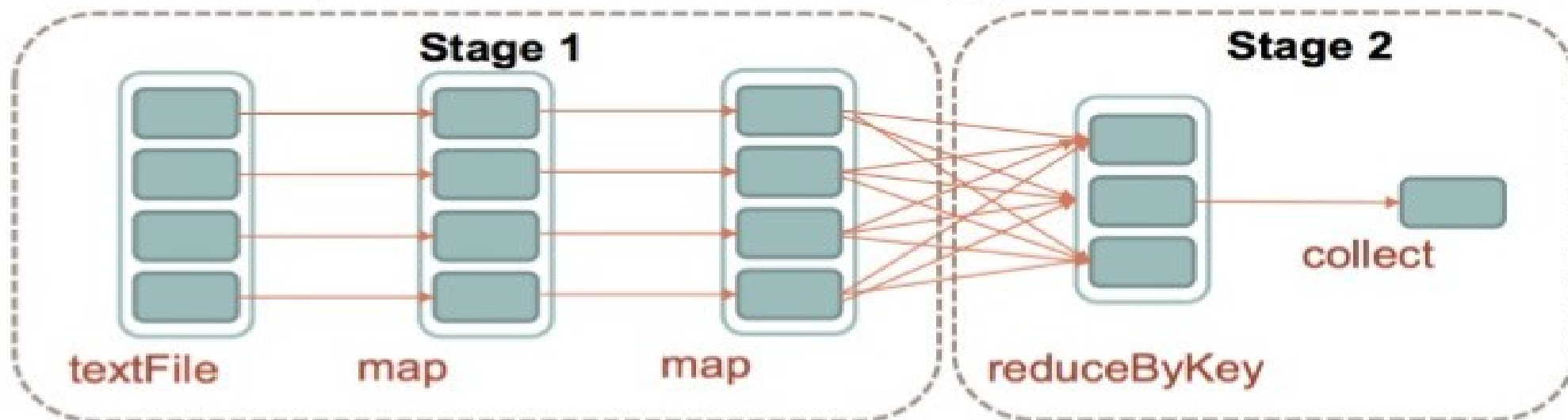
- `scala> val data = Array(25, 20, 15, 10, 5)`
 - `data: Array[Int] = Array(25, 20, 15, 10, 5)`
- `scala> val distData = sc.parallelize(data)`
 - `distData: org.apache.spark.rdd.RDD[Int] = ParallelCollectionRDD[0] at parallelize at <console>:14`
- `scala> distData.reduce(_ + _)`
 - `res0: Int = 75`

- `scala> val distFile = sc.textFile("README.md")`
 - `distFile: org.apache.spark.rdd.RDD[String] = MappedRDD[3] at textFile at <console>:12`
- `scala> distFile.first`
 - `res5: String = #Apache Spark`

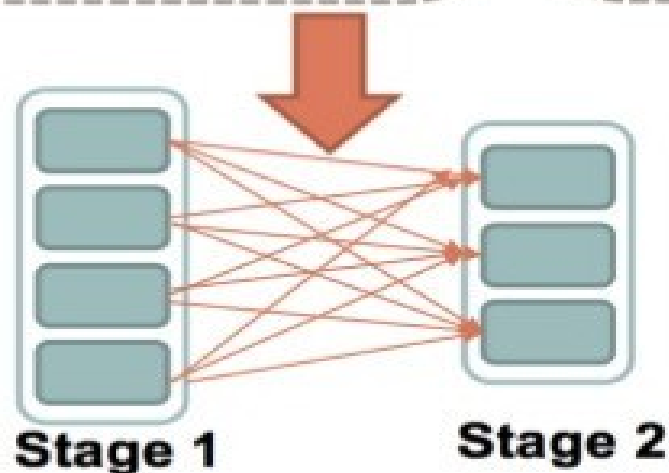
- `scala> val distFile = sc.textFile("hdfs://mi_carpeta:9000/README.txt")`
 - `distFile: org.apache.spark.rdd.RDD[String] = MappedRDD[5] at textFile at <console>:12`
- `scala> distFile.first`
 - `res2: String = For the latest information about Hadoop, please visit our website at:`

- `val file = sc.textFile("hdfs://...")`
- `val counts = file.flatMap(line => line.split("")).map(word => (word, 1)).reduceByKey(_ + _)`
- `counts.saveAsTextFile("hdfs://....")`

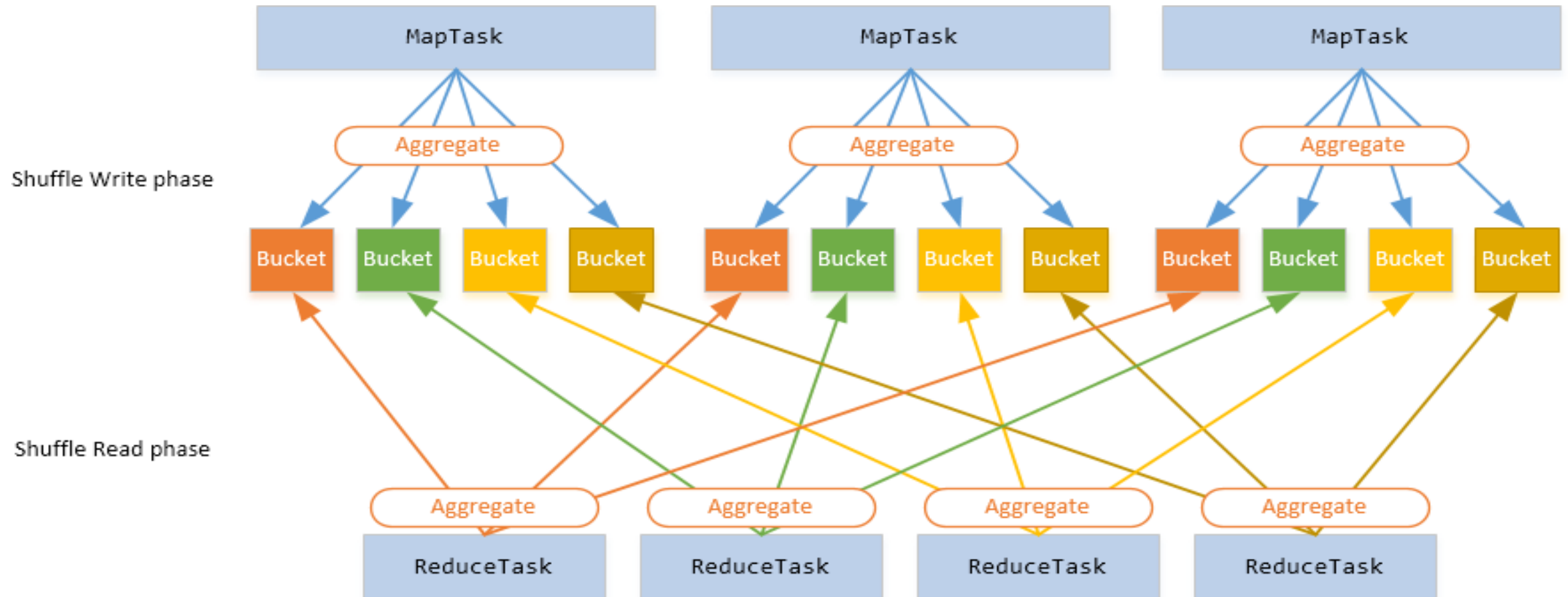
How Spark Works - Stages



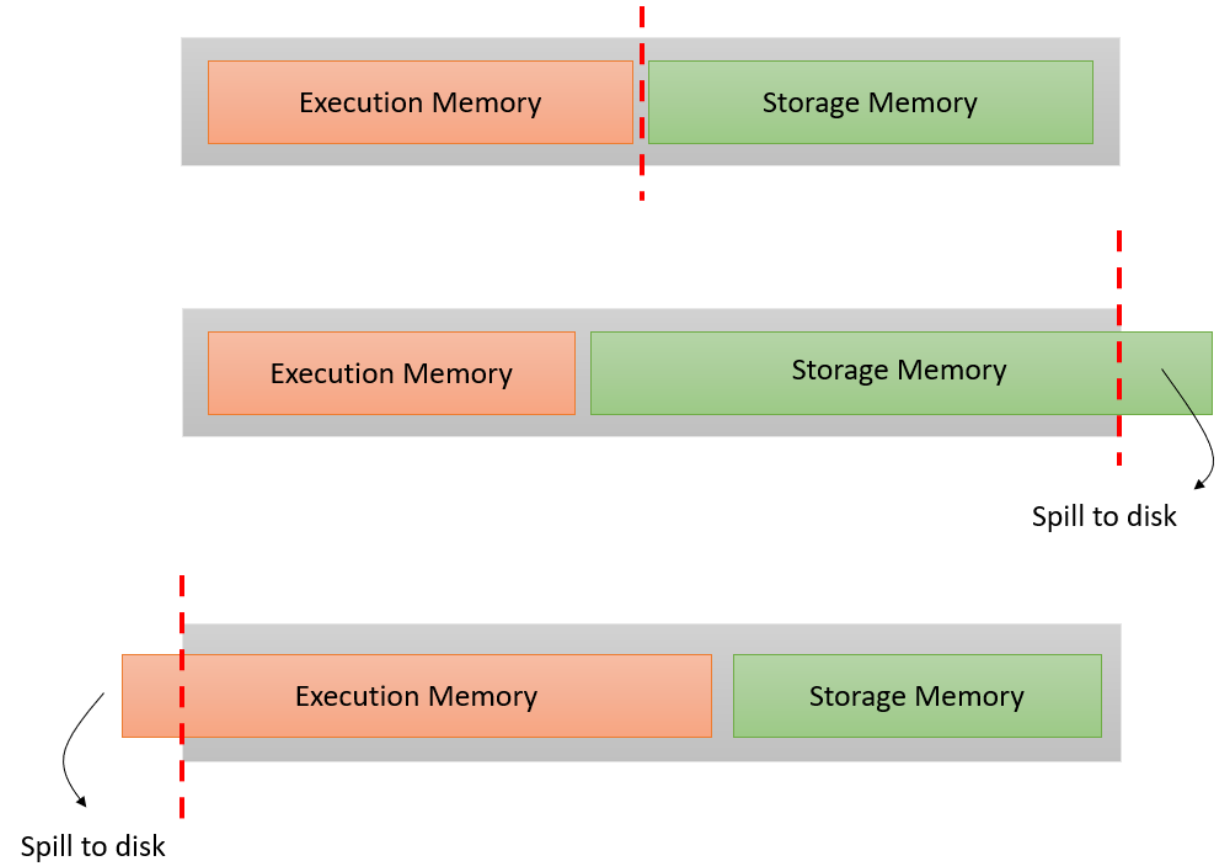
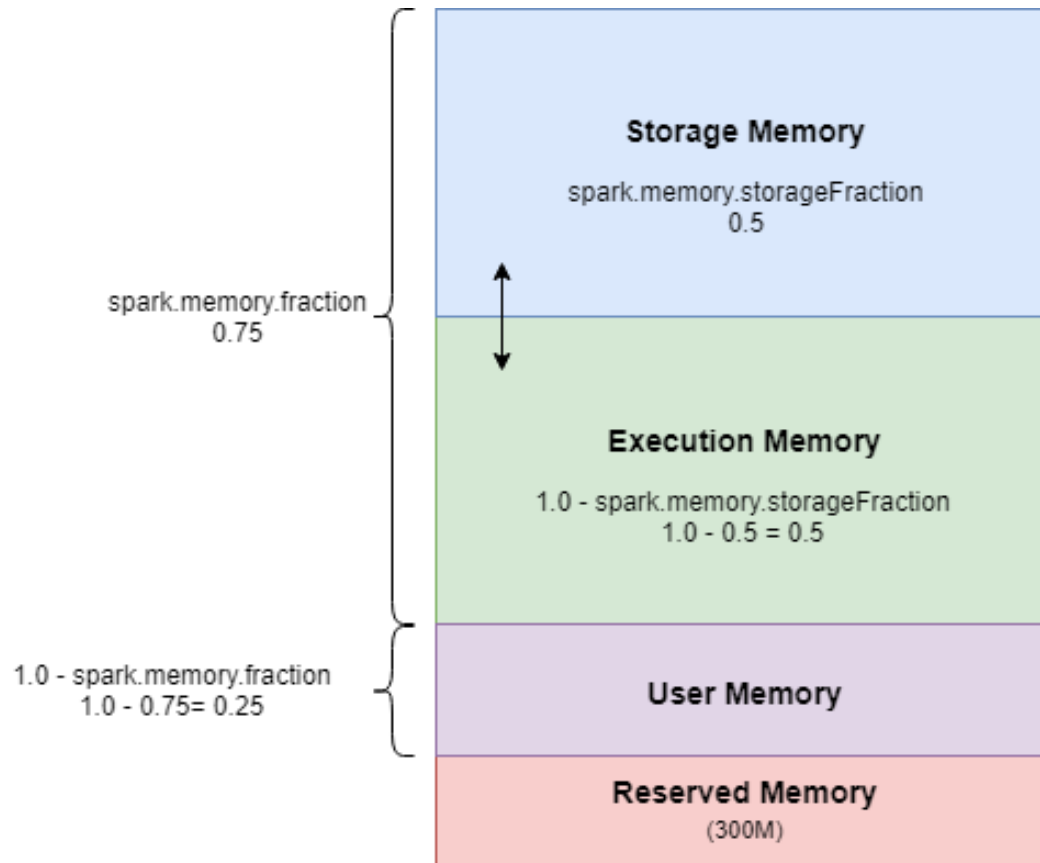
DAG (Directed Acyclic Graph).



Each stage is executed as a series of Task (one Task for each Partition).



Memory Storage





 Calle Playa de Liencres, 2 bis
(entrada por calle Rozabella)
Parque Europa Empresarial
Edificio Madrid
28290 Las Rozas, Madrid

 900 373 379  info@u-tad.com

 [SOLICITA MÁS INFORMACIÓN](#)



CENTRO ADSCRITO A:



PROYECTO COFINANCIADO POR:

